

EC8202 – Big Data Analytics Mini-Project

FLEET LAMBDA PLATFORM

Senevirathne P.U.S	EG/2021/4805
Threemavithana T.M.	EG/2021/4835
Kodikara A.W.	EG/2021/4613

Repository: <https://github.com/udaraDev/fleet-lambda-platform.git>

1 Use case and requirements

A ride-hailing operator needs immediate visibility into fleet utilization and earnings while reconciling completed trips against daily fuel and maintenance costs. The two views have different truth requirements: live state may be indicative, while daily profit must not treat missing telemetry or missing costs as zero.

1.1 Business questions

- What is current fleet utilization, idle ratio, trip activity and earnings by zone and time of day?
- Which vehicles are genuinely unprofitable after yesterday's costs, and which results remain unknown because evidence is incomplete?
- Can operators detect stalled ingestion, failed reconciliation, missing dates, damaged exports and bad source rows?

1.2 Interpreted requirements

ID	Requirement	Acceptance evidence
R1	Continuous Python telemetry through Kafka partitions	Checkpointed Spark query and fresh-install smoke test
R2	One expense file per simulated day	Producer creates dated CSV; Airflow discovers closed days
R3	Meaningful cleaning, enrichment, aggregation and join	Native validation, time bucket, trip aggregate, coverage and cost join
R4	Queryable serving and consolidated result	PostgreSQL, FastAPI, JSON report and business results page
R5	Observable health and failures	Structured JSON logs; ingestion and report-integrity HTTP rules
R6	Reproducible local execution	Compose, migrations, unit/integration tests and empty-volume verification

The persistent simulation clock starts at 2026-03-01 00:00 UTC. One simulated day equals 300 real seconds. A telemetry tick occurs every two real seconds and advances 9.6 simulated minutes. Times are UTC and monetary values are stored as integer LKR cents.

2 Architecture decision: Lambda vs Kappa

Lambda (introduced by Nathan Marz) was selected because corrected daily cost files must restate historical accounts while immutable trip evidence remains replayable. The speed path gives operational visibility; the batch path recomputes a vehicle-day result from committed archives and the latest valid expense file.

Decision factor	Lambda in this project	Kappa alternative
Latency	Live state and lookbacks update every micro-batch.	A single stream could also meet low latency.
Replay	Spark batch rereads verified Parquet by day.	Both expenses and telemetry must be durable versioned events.
Consistency	Daily output is restated from a fixed input snapshot and marked by run/version.	Historical keyed state needs correction/retraction semantics.
Cost/complexity	Two code paths but simple local recovery and explainable accounting.	One topology in principle, but more state/replay operational complexity here.
Failure isolation	Live metrics can continue when one expense date fails.	A unified topology can couple operational and accounting failures.

2.1 Rejected alternative

Kappa (popularized by Jay Kreps) was rejected for this two-week local platform, not because Kappa is inferior in general. It becomes attractive when all sources are naturally versioned event streams and the team can operate replay, state migration and retractions. Daily CSV corrections make the separate batch truth path easier to verify and defend.

2.2 Trade-off

Lambda duplicates identity and transformation logic. This risk is mitigated by a shared fixed event-field contract, UTC timestamp canonicalization, parity tests, persistent identities, deterministic daily recomputation and explicit completeness states. Eventual consistency remains: a late correction is visible after the next Airflow restatement.

Consistency promise: no end-to-end exactly-once claim. Kafka checkpoints, event identities, trip keys, archive manifests, database transactions and idempotent restatement provide practical replay safety within the documented single-writer dataset contract.

3 Architecture evolution and data flow

3.1 Earlier design - superseded

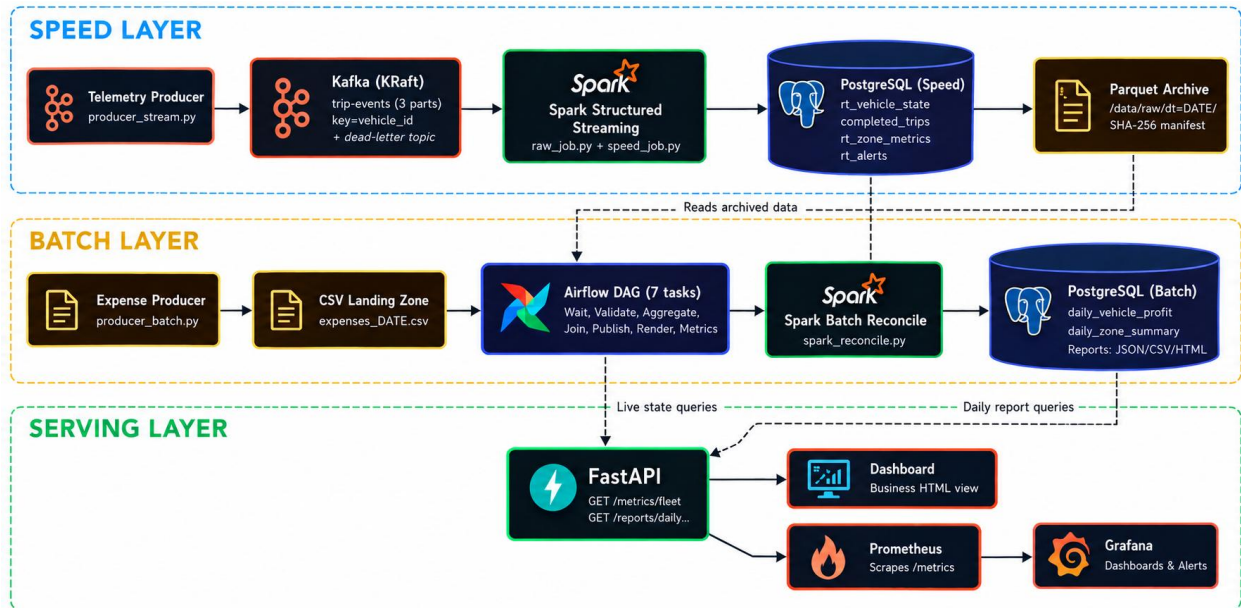


Figure 1. Earlier architecture diagram, retained with a white background. This records the original seven-stage Airflow proposal, not the final deployed workflow.

The initial design separated waiting, validation, aggregation, joining, publication, rendering and metrics into seven Airflow tasks. The implementation consolidates these responsibilities into one orchestration task in the daily profitability workflow; a separate workflow performs data-quality checks. Spark still performs the batch aggregation and expense join.

How to read the earlier diagram: The local archive path and PostgreSQL-to-archive arrow belong only to the superseded proposal. Figure 2 shows the implemented design, in which Kafka feeds independent speed and raw Spark consumers, and the raw consumer writes Parquet directly to MinIO.

3.2 Final architecture overview

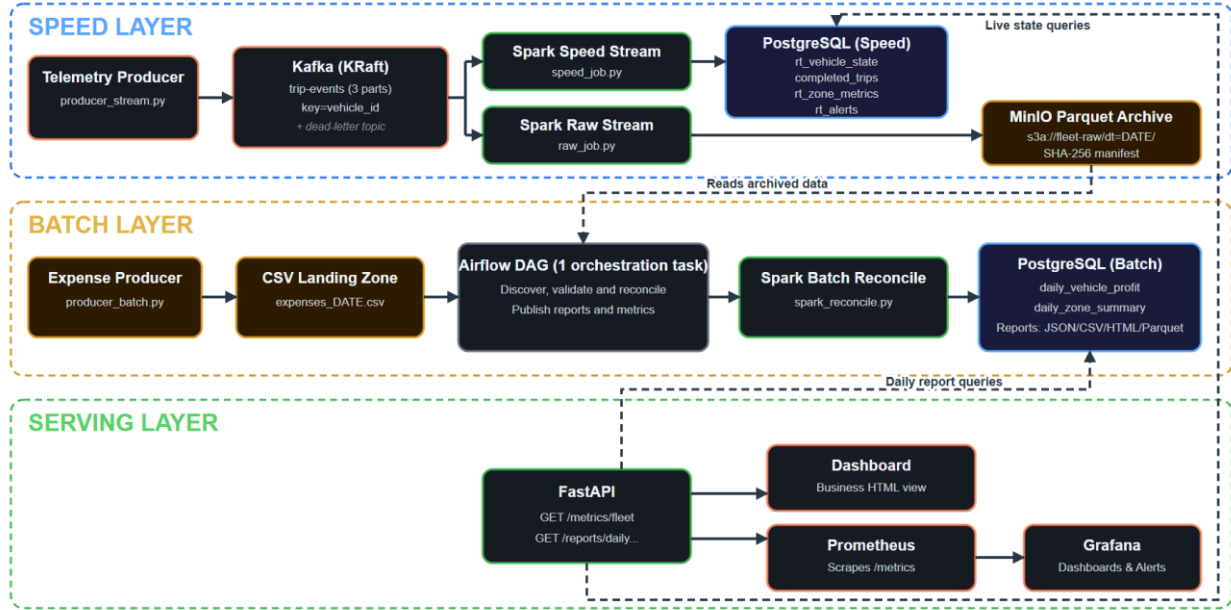


Figure 2. Final implemented Lambda architecture. Kafka feeds independent Spark speed and raw consumers. The speed stream writes operational state and event-time metrics to PostgreSQL, while the raw stream writes immutable Parquet files and integrity manifests directly to MinIO. Airflow orchestrates Spark batch reconciliation with daily expense data. FastAPI serves live and historical results, while Prometheus and Grafana provide monitoring and visualisation.

The speed query validates Kafka records, updates live state and computes one-minute event-time windows with a two-minute watermark. Per-event metric contributions allow the serving transaction to retract a discredited identity or trip and protect the corrected window from a later non-retracting Spark update. The raw query independently archives valid events to MinIO and publishes checksum manifests. A session advisory lock fences archive writes; a transaction lock protects serving commits. Airflow invokes Spark batch reconciliation independently for each ready date, newest first, with a configurable five-changed-date budget.

Storage contracts: Raw MinIO Parquet is accepted only through committed row-count and SHA-256 manifests. PostgreSQL stores live state, trips, identities, conflicts, quarantine, run history and daily profit. Every published JSON, CSV, HTML and Parquet representation is covered by one committed digest manifest.

4 Technology selection

Component	Why selected	Constraint / rejected alternative
Kafka 3.9	Required ingestion technology; keyed events and three partitions demonstrate partitioned transport.	Single broker is suitable only for a laptop; no replication.
Spark 3.5 Structured Streaming	Required processing choice; native schema validation and checkpointed Kafka consumption.	Storm rejected to keep one engine across stream and batch.
Spark DataFrames (batch)	Distributed trip aggregation, conflict detection, coverage and expense join.	Pure Python dictionaries were removed from production batch processing.
Airflow 2.10	Scheduled discovery, retries, UI and serialized daily reconciliation.	One visible task is simpler but offers less stage-level retry detail.
PostgreSQL 16	Transactions, constraints, JSON metadata and queryable serving.	Cassandra is less suited to relational reconciliation and local transactions.
MinIO + Parquet	S3-compatible immutable replay source with typed timestamps, date partitions and verified manifests.	Single-node MinIO is for local reproducibility, not high availability.
FastAPI	Typed validation, API documentation, health endpoints and a small business view.	Airflow UI is not a fleet dashboard.
Docker Compose	Repeatable single-host wiring and persistent named volumes.	Not production orchestration.

The implementation pins major Python dependencies and container images. Host tests use Python 3.11 where possible; CI installs the development requirements and validates Compose configuration. Local demo credentials and loopback-only ports are not suitable for external deployment.

5 Implementation

5.1 Streaming path

- Python emits telemetry for twelve vehicles; every fourth vehicle remains parked to create an interpretable idle/cost-only case.
- Both consumers use `from_json` plus native column constraints. Unknown vehicles, malformed JSON, impossible coordinates/speed, invalid types and out-of-bounds simulated timestamps are rejected.
- Kafka is capped at 2,000 offsets per trigger. Spark executors bulk-stage valid partitions and PostgreSQL performs a set-based transactional merge; raw event rows and aggregate windows are not collected on the Python driver.
- Raw commits are identified by canonical Kafka partition/offset ranges, not restartable Spark batch numbers. Transactional per-partition high-water marks reject both gaps and overlaps, so recreated checkpoints cannot silently collide with an earlier ledger generation.
- Persistent event fingerprints deduplicate across batches. Same identity with different business content becomes a conflict; implicated trip revenue and live state are removed rather than arbitrarily selected.

5.2 Batch path

- Only ledger-committed and checksum-verified Parquet enters reconciliation. Spark detects event/trip conflicts, aggregates completions by vehicle and calculates telemetry coverage.
- All registered vehicles remain visible. Complete telemetry plus expenses yields profit; missing telemetry, conflicts or missing expenses yields a null profit with a quality status.
- A greater-than-five-percent expense rejection rate fails that date without replacing the last good result. Other dates continue and the Airflow task reports a failure summary.
- Database rows move to pending before file export. Atomic file replacement then publishes a digest. Missing/corrupt published files are automatically regenerated.

5.3 Serving semantics

The live fleet endpoint uses a latest-state simulated-time lookback. The zone endpoint accepts a 1-1440 minute window and reads Spark event-time rows, including conflict-retracted windows. Both are explicitly labelled indicative. The daily endpoint returns vehicle rows and publication metadata from one SQL statement, avoiding mismatched versions during concurrent publication. The browser page refreshes every 15 seconds and presents LKR values, report quality and algorithm version.

6 Observability and failure behavior

Signal	Detection	Response / meaning
Live/raw freshness	Accepted-event age plus live-to-archive event-time lag	HTTP 503 for stale ingestion or raw lag beyond 60 simulated minutes
Batch failures	Latest run status per date and stalled-run threshold	HTTP 503; failed date retained with error
Date completeness	Every expected date from simulation start to latest closed day	Missing historical hole makes report health degraded
Publication	Pending state, algorithm version and four-file SHA-256 manifest	Missing/corrupt/outdated report makes health degraded and triggers restatement
Data quality	Quarantine rows and >5% daily gate	Bad date fails; last good daily result remains
Tracing	trace_id in source/live trip records and structured logs	Supports event walkthrough; not full distributed tracing
Business alert	Observed idle_since against simulated latest event	API lists considerably idle vehicles

Structured JSON logs include timestamp, stage and message plus batch/run identifiers, row counts and errors where relevant. The official Prometheus Python client exports metrics; a pinned Prometheus server evaluates four local rules, including raw/archive divergence, and a provisioned Grafana dashboard visualises pipeline health. External notification routing is deliberately outside the local submission.

6.1 Failure exercise

Final verification stopped only the telemetry producer. Ingestion health returned HTTP 503 at a 126-second event age, then the cleanup handler restarted the producer and health recovered to HTTP 200 at 5.6 seconds. The demo runbook repeats this exercise. Because the simulation wall clock continues, an outage creates an honest historical coverage gap; the system must not fabricate events or profit.

Recovery rule: PostgreSQL, Kafka, Parquet and Spark checkpoints form one logical dataset. Raw commits use offset fingerprints; if a raw checkpoint is recreated, retained Kafka can repair an event-time gap without reusing Spark batch IDs. Conflicts still require audited source correction rather than ad-hoc deletion.

7 Results and business output

The running platform exposes a consolidated business page at <http://localhost:8001>. It combines current reporting/active vehicles, idle ratio, hourly earnings, zone activity and a selectable daily vehicle profitability table. It also states the report run, publication state, algorithm version and quality outcome.

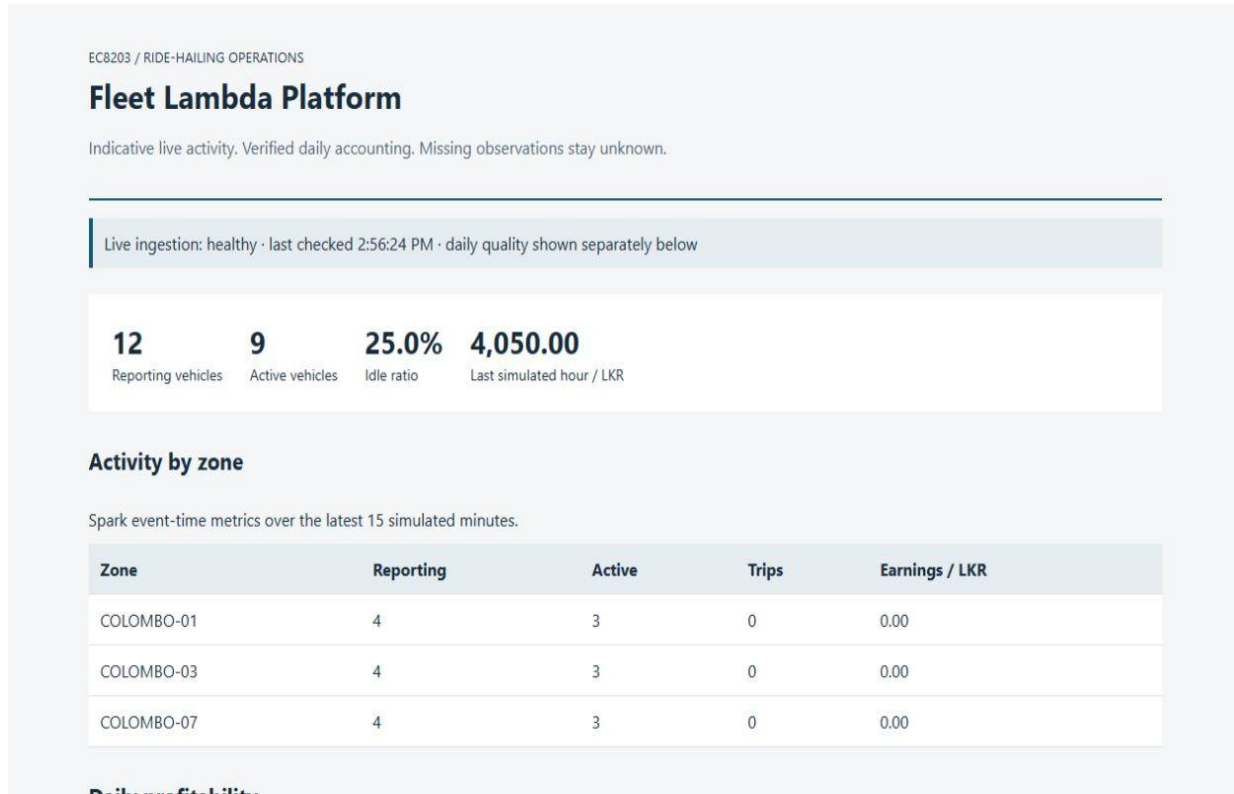


Figure 3. Actual local results page showing healthy ingestion, the live fleet summary and the parameterised Spark event-time zone view. The selectable daily table continues below the captured viewport.

7.1 Verified snapshots

Evidence	Observed result
Automated unit/API/archive tests	Host run: 65 tests passed and one dependency-gated Spark test was skipped. Docker run with the complete Spark dependencies: all 66 tests passed, including the Spark-to-Python parity test.
Isolated Spark/PostgreSQL suite	26 named checks passed; disposable schema and isolated MinIO prefix
Fresh volumes	Eighteen-service Compose definition; DAG imports, report, live, MinIO and monitoring checks
Short throughput smoke test	10/100/500 target eps: all 30/300/1,500 events accepted; p95 latency 8.48/6.03/6.15 s
Final report-integrity health	At evidence capture, all then-closed reports were valid; live health permits one current-date reconciliation in progress
Final result quality snapshot	3,512 complete vehicle-days; 5,500 incomplete-telemetry vehicle-days

The historical counts are a dated snapshot, not a performance benchmark. Incomplete days reflect real downtime in the persistent demonstration dataset. All retained dates were restated under algorithm version 4 so export digests and identity semantics were recalculated rather than inherited.

7.2 Verification and reproducibility

The verification strategy separates pure logic, API behaviour, isolated database/Spark integration, live smoke checks and a fresh-volume installation. Faults that would damage evidence are applied only to temporary directories and UUID-named schemas.

Area	Checks
Replay/integrity	Cross-batch duplicate, offset identity, timestamp parity, conflicting fare, archive loss, writer fence
Financial truth	Trip counts, zero-revenue parked vehicle, corrected cost, missing expense, incomplete telemetry, profit equation
Publication	Pending state, simulated replace failure, retry, missing/corrupt four-format export recovery, digest/run match
Validation	Malformed JSON, unknown vehicle, future timestamp and daily cost quality gate
Concurrency	Same-date Airflow lock and consistent one-statement API read
Operations	Compose schema, DAG imports, all endpoints, empty PostgreSQL/Kafka/archive/checkpoint volumes

7.3 Fresh-install evidence

An isolated Compose project used empty uniquely named volumes, alternate localhost ports and the already built images. It generated live data and a daily report, passed the read-only smoke test, showed no Airflow import errors, and was then stopped. Its disposable volumes were removed after machine-readable evidence was saved. This validates initialization on the same Docker host; it does not claim an uncached second-machine download test.

7.4 Reproduction

- Run `python -m scripts.bootstrap_env` once to create random local secrets, then run `docker compose up --build -d`.
- Run `python -m unittest discover -s tests -v` and `python -m scripts.verify_running --wait-seconds 480`.
- Run the isolated reconciliation verification in the application image; it cleans only its UUID schema.
- Use `scripts/verify_clean_install.py` when its alternate ports are free; it records evidence, stops containers and removes only its uniquely named disposable volumes.

8 Limitations, scale and security

Current limitation	Production direction
Single Kafka broker / local[2] Spark	Replicated Kafka, distributed Spark and capacity testing
Single PostgreSQL staging/merge target	Partitioned staging tables or a distributed serving sink after measured load
Small immutable MinIO Parquet files	Iceberg/Delta compaction, lifecycle policies and replicated object storage
One Airflow task	Stage-specific tasks/sensors after operational need is demonstrated
Two-minute watermark is a classroom policy	Tune allowed lateness from measured production arrival distributions
Generated local credentials; no API auth/TLS	Externally managed secrets, authentication, TLS and network policy
Local Prometheus/Grafana; no routed notifications	Production SLOs, Alertmanager routes and on-call ownership
Manual conflict resolution	Audited correction/rebuild workflow and lineage tooling
Short 500 eps smoke test only	Long-duration saturation, resource profiling and realistic arrival distributions

The project implements MinIO, separate consumers, formal windows, Grafana and durable alert history. It still treats them as local teaching infrastructure: correctness, architecture reasoning and reproducible evidence remain more important than naming production tools.

8.1 Security scope

Host ports bind to loopback. Each startup generates independent local secrets; FastAPI has a read-only database role, stream and batch writers have separate roles, and MinIO applications use a bucket-scoped identity. Before external deployment, move secrets to a manager, authenticate endpoints, enable TLS and define retention/privacy controls for location traces.

8.2 Ethical interpretation

Profitability is an operational vehicle measure, not a driver-performance score. Missing data and conflicts are shown as unknown to reduce harmful inference. A production system would require governance for location access, retention and human review of alerts.

9 Conclusion

The delivered platform meets the core mini-project objective: two simulated sources feed an observable Lambda pipeline; independent Spark consumers provide raw and speed layers; Spark also performs historical reconciliation; Airflow orchestrates profitability and data-quality work; results land in queryable PostgreSQL and versioned exports; and FastAPI, Prometheus and Grafana expose operational and financial answers.

The strongest engineering decision is not a particular tool but the treatment of uncertainty. Duplicate identities are idempotent, conflicts invalidate authority, missing telemetry does not become zero revenue, bad cost files preserve the last good result, and published files are checked rather than assumed. The result is a defensible teaching system with explicit boundaries.

10 References

- [1] N. Marz and J. Warren, *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Shelter Island, NY, USA: Manning Publications, 2015.
- [2] J. Kreps, “Questioning the Lambda Architecture,” *O’Reilly Radar*, Jul. 2, 2014. [Online]. Available: <https://www.oreilly.com/radar/questioning-the-lambda-architecture/>. [Accessed: Sep. 26, 2026].
- [3] Apache Software Foundation, “Apache Kafka 3.9 Documentation.” [Online]. Available: <https://kafka.apache.org/39/>. [Accessed: Sep. 27, 2026].
- [4] Apache Software Foundation, “Spark 3.5.6 Structured Streaming Programming Guide.” [Online]. Available: <https://spark.apache.org/docs/3.5.6/structured-streaming-programming-guide.html>. [Accessed: Sep. 28, 2026].
- [5] Apache Software Foundation, “Apache Airflow 2.10.5 Documentation.” [Online]. Available: <https://airflow.apache.org/docs/apache-airflow/2.10.5/>. [Accessed: Sep. 28, 2026].
- [6] PostgreSQL Global Development Group, “PostgreSQL 16 Documentation.” [Online]. Available: <https://www.postgresql.org/docs/16/>. [Accessed: Sep. 27, 2026].
- [7] FastAPI, “FastAPI Documentation.” [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed: Sep. 28, 2026].

11 Individual contributions

Member	Individual work area
Threemavithana T.M.	Platform, Kafka/MinIO ingestion and monitoring
Senevirathne P.U.S	Spark streaming, PostgreSQL serving and API
Kodikara A.W.	Airflow/Spark batch, data quality and reports